

One-Period Binomial Option Pricing

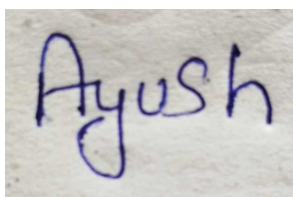
Live-Data Implementation and Streamlit Dashboard

Group 4 — Instructor: Prof. Mithun Radhakrishna

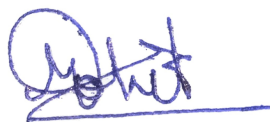
Name	Contribution
Vansh Goel	Website (Streamlit) Building
Ayush Kumar	Colab / Python Code Building
Yug Mitulkumar Desai	Report Writing
Mohit Kumar Yadav	Presentation (PPT) Making

Date: 21st April 2026

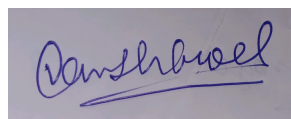
Member Signatures:



(a) Ayush Kumar



(b) Mohit Kumar Yadav



(c) Vansh Goel



(d) Yug M Desai

Abstract. This report documents our implementation of the one-period binomial option pricing model for a European option, fulfilling the tasks set in Group 4 of ES 418. The system fetches live stock and option-chain data from Yahoo Finance, computes realized volatility, constructs a one-period binomial tree, calculates option payoffs, determines the replicating portfolio (Δ, B) , and prices the option via two equivalent methods — replication and risk-neutral valuation. An interactive Streamlit dashboard visualizes all results in real time. We demonstrate the complete workflow using META Platforms, Inc. as a live case study.

Colab Notebook Link: [\[Colab Link Here\]](#)

Streamlit App Link: [binomial-model.streamlit.app](#)

Website Code Link: [\[GitHub Link Here\]](#)

1. Introduction and Motivation

Options are derivative contracts giving the holder the right, but not the obligation, to buy (call) or sell (put) an underlying asset at a pre-agreed *strike price* K on or before a specified expiry date. Pricing such contracts requires a model of how the underlying asset price can evolve. The **one-period binomial model**, introduced by Cox, Ross, and Rubinstein (1979), is the simplest arbitrage-free framework: it assumes that the stock price moves to exactly one of two states at time T .

Our project implements this model end-to-end with live market data, covering all four tasks specified in the assignment:

1. Construct a one-period binomial tree.
2. Calculate option payoffs in the up and down states.
3. Determine the replicating portfolio (Δ, B) .
4. Compute the option price.

Reference for Cox Ross and Rubinstein (1979)

Cox, J. C., Ross, S. A., & Rubinstein, M. (1979). *Option pricing: A simplified approach*. Journal of Financial Economics, **7**(3), 229–263.
DOI: 10.1016/0304-405X(79)90015-1

2. Mathematical Framework

2.1. Binomial Tree Construction (Task 1)

Given current spot price S_0 , we model price evolution over one period of length T (in years) using factors u (up) and d (down) derived from the *annualized volatility* σ :

$$u = e^{\sigma\sqrt{T}}, \quad d = e^{-\sigma\sqrt{T}} = \frac{1}{u} \quad (1)$$

The two possible end-of-period prices are:

$$S_u = S_0 u, \quad S_d = S_0 d \quad (2)$$

No-arbitrage condition: For the model to be arbitrage-free, the risk-free growth factor e^{rT} must lie strictly between d and u :

$$d < e^{rT} < u \quad (3)$$

The code enforces this check and raises an error if violated.

2.2. Option Payoffs (Task 2)

For a European **call** with strike K :

$$C_u = \max(S_u - K, 0), \quad C_d = \max(S_d - K, 0) \quad (4)$$

For a European **put**:

$$P_u = \max(K - S_u, 0), \quad P_d = \max(K - S_d, 0) \quad (5)$$

2.3. Replicating Portfolio (Task 3)

The replicating portfolio consists of Δ shares of stock and B dollars in a risk-free bond, chosen so that the portfolio matches the option payoff in both states:

$$\Delta \cdot S_u + B e^{rT} = C_u \quad (\text{up state}) \quad (6)$$

$$\Delta \cdot S_d + B e^{rT} = C_d \quad (\text{down state}) \quad (7)$$

Solving the two equations:

$$\boxed{\Delta = \frac{C_u - C_d}{S_u - S_d}} \quad \boxed{B = e^{-rT}(C_u - \Delta \cdot S_u)} \quad (8)$$

Δ is the **option delta** (hedge ratio), and B is the **bond position** (present value of the cash leg). For a put, $\Delta < 0$ (short stock) and $B > 0$ (long bond).

2.4. Option Price — Two Equivalent Methods (Task 4)

Method 1 — Replication:

$$\boxed{V_0 = \Delta \cdot S_0 + B} \quad (9)$$

Method 2 — Risk-Neutral Pricing:

Define the risk-neutral probability:

$$p^* = \frac{e^{rT} - d}{u - d} \quad (10)$$

Then:

$$\boxed{V_0 = e^{-rT}[p^* C_u + (1 - p^*) C_d]} \quad (11)$$

Both methods must agree exactly (the code asserts this with a tolerance of 10^{-8}), confirming the model's internal consistency.

2.5. Realized Volatility

Rather than using an implied or assumed volatility, the model computes σ from historical data. Given a window of N daily log returns $r_i = \ln(S_i/S_{i-1})$:

$$\sigma_{\text{daily}} = \text{std}(r_1, \dots, r_N), \quad \sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252} \quad (12)$$

The code uses a default rolling window of the last 60 trading days.

3. Code Architecture and Walkthrough

3.1. High-Level Flow

The figure below summarises the complete execution flow of `binomial_model.py` and maps it to the four assignment tasks.

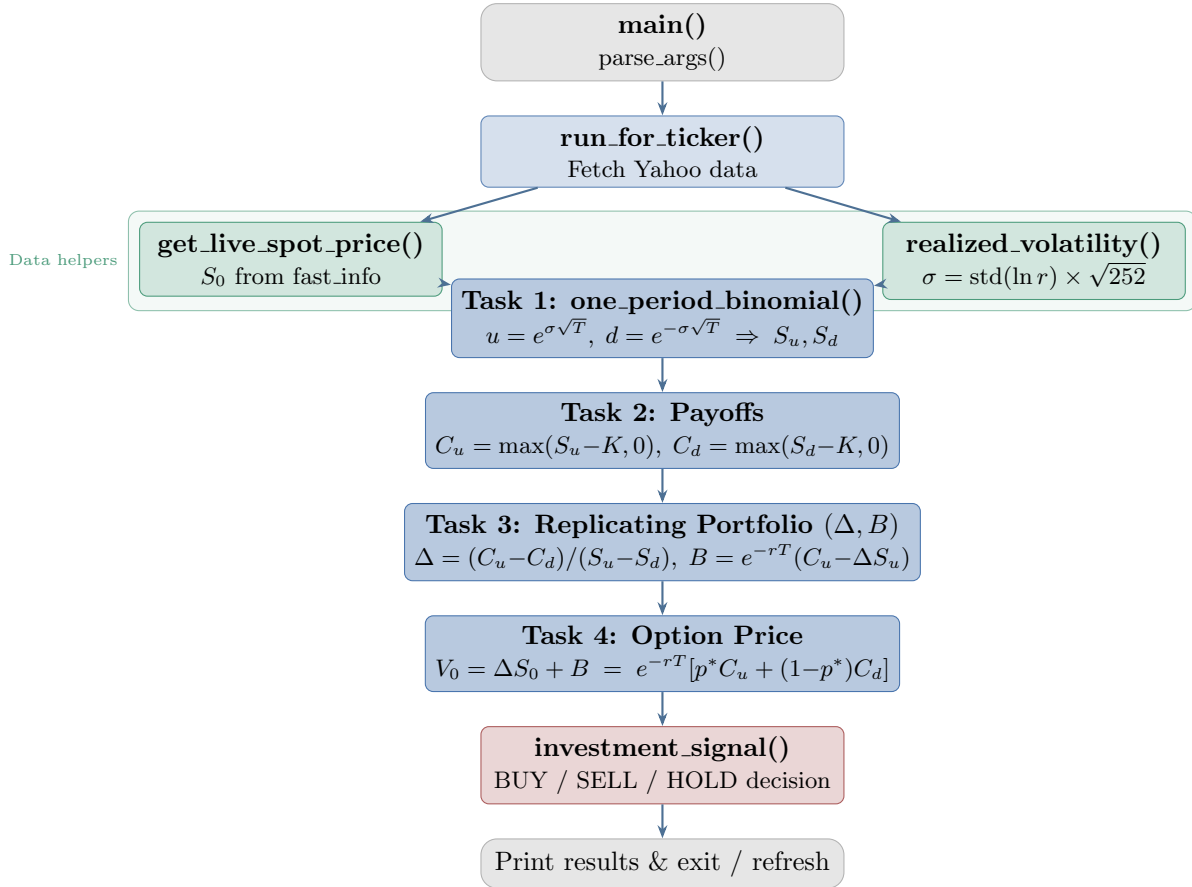


Figure 2: Execution flowchart of the one-period binomial pricing engine.

3.2. Key Functions

`realized_volatility()` computes σ from log returns over a rolling window. The standard deviation is multiplied by $\sqrt{252}$ to annualise.

Listing 1: Annualised realized volatility

```

1 def realized_volatility(close_prices, window=60):
2     log_returns = np.log(close_prices / close_prices.shift(1)).dropna()
3     log_returns = log_returns.tail(window)
4     sigma_daily = float(log_returns.std(ddof=1))
5     return sigma_daily * np.sqrt(252.0)    # annualise

```

`realized_volatility_last_minutes()` fetches the last N minutes of 1-minute intraday bars and computes a short-horizon annualised volatility. The annualization factor differs from the daily case because the base interval is one minute rather than one trading day:

$$\sigma_{\text{annual}}^{\text{intraday}} = \sigma_{1\text{m}} \times \sqrt{252 \times 6.5 \times 60} = \sigma_{1\text{m}} \times \sqrt{98,280} \quad (13)$$

where $\sigma_{1\text{m}}$ is the standard deviation of 1-minute log returns over the chosen window. An intermediate one-hour volatility $\sigma_{1\text{h}} = \sigma_{1\text{m}}\sqrt{60}$ is also returned for diagnostic display on the dashboard.

Listing 2: Intraday (1-minute) realized volatility

```

1 def realized_volatility_last_minutes(symbol, interval="1m",
2                                     period="5d", window_minutes=60):
3     raw = yf.download(symbol, period=period, interval=interval,
4                       auto_adjust=True, progress=False)
5     close = extract_close_series_from_download(raw, symbol)
6     close_tail = close.tail(window_minutes) # last N bars
7
8     r_1m = np.log(close_tail / close_tail.shift(1)).dropna()
9     sigma_1m = float(r_1m.std(ddof=1))
10    sigma_1h = sigma_1m * np.sqrt(60.0) # 1-hour vol
11    minutes_per_year = 252.0 * 6.5 * 60.0
12    sigma_annual = sigma_1m * np.sqrt(minutes_per_year)
13    return sigma_annual, sigma_1h, close_tail

```

`choose_strike()` and `choose_expiry()` handle automatic contract selection from the live option chain. `choose_strike` finds the strike closest to the current spot price using the ℓ_1 nearest-neighbour rule:

$$K^* = \arg \min_{K_i \in \mathcal{K}} |K_i - S_0| \quad (14)$$

making it an *at-the-money* (ATM) selector by default. `choose_expiry` returns the nearest available expiry from the live option chain when the user does not specify one.

Listing 3: ATM strike and nearest-expiry selection

```

1 def choose_strike(chain_df, spot, user_strike=None):
2     strikes = chain_df["strike"].dropna().astype(float).values
3     if user_strike is None:
4         return float(strikes[np.argmin(np.abs(strikes - spot))])
5     return float(strikes[np.argmin(np.abs(strikes - user_strike))])
6
7 def choose_expiry(ticker, user_expiry=None):
8     expiries = list(ticker.options) # sorted nearest-first
9     if user_expiry is None:
10        return expiries[0] # default: nearest expiry
11    return user_expiry

```

`one_period_binomial()` implements all four tasks. First, the up/down factors are built, then payoffs are computed, followed by the replicating portfolio and the option price via both methods, with a consistency check:

Listing 4: Core binomial pricing function (abridged)

```

1 def one_period_binomial(spot, strike, r, maturity_years, sigma,
2                         option_type):
3     u = np.exp(sigma * np.sqrt(maturity_years)) # Task 1
4     d = np.exp(-sigma * np.sqrt(maturity_years))
5     su, sd = spot * u, spot * d
6
7     payoff_up = max(su - strike, 0.0) # Task 2 (call)
8     payoff_down = max(sd - strike, 0.0)
9
10    delta = (payoff_up - payoff_down) / (su - sd) # Task 3
11    bond = np.exp(-r * maturity_years) * (payoff_up - delta * su)
12
13    p = (np.exp(r * maturity_years) - d) / (u - d) # Task 4
14    price_rn = np.exp(-r * maturity_years) * (p*payoff_up + (1-p)*
15        payoff_down)
16    price_rep = delta * spot + bond

```

```

15     assert abs(price_rn - price_rep) < 1e-8           # must agree
16     return BinomialResult(...)

```

`extract_market_option_price()` infers the fair market price of the option from the live option chain using a priority rule: The *bid-ask midpoint* is preferred when both quotes are available (as it is a better estimator of fair value than the last-trade price, which may be stale):

$$V_0^{\text{market}} = \begin{cases} \frac{\text{bid} + \text{ask}}{2} & \text{if bid} > 0 \text{ and ask} > 0 \\ \text{lastPrice} & \text{otherwise (fallback)} \end{cases} \quad (15)$$

Listing 5: Market price from bid-ask midpoint or last trade

```

1 def extract_market_option_price(chain_df, strike):
2     row = chain_df.loc[(chain_df["strike"] - strike).abs().idxmin()]
3     bid = float(row.get("bid", 0.0) or 0.0)
4     ask = float(row.get("ask", 0.0) or 0.0)
5     last = float(row.get("lastPrice", np.nan))
6     if bid > 0 and ask > 0:
7         return (bid + ask) / 2.0      # midpoint preferred
8     if np.isfinite(last) and last > 0:
9         return last                  # fallback to last trade
10    raise ValueError("Could not infer market option price.")

```

Maturity horizon selection in `run_for_ticker()` offers two modes, toggled via the Streamlit sidebar or the `--one-hour-step` flag:

- **Expiry horizon (default):** T is computed as the exact calendar time remaining to the option's expiry date,

$$T = \frac{(\text{expiry_datetime} - \text{now}) \text{ in seconds}}{365 \times 24 \times 3600}$$

For META on 20 April 2026 with expiry 2026-05-08, this gives $T = 0.04689$ years (≈ 17 calendar days).

- **One-hour step:** T is fixed at one trading hour,

$$T = \frac{1}{252 \times 6.5} \approx 6.09 \times 10^{-4} \text{ years}$$

This shrinks the binomial spread dramatically ($u \approx 1 + \sigma\sqrt{T}$ for small T) and is useful for intraday signal generation.

Listing 6: Dual maturity horizon modes in `run_for_ticker()`

```

1 t_to_expiry = (expiry_date - now).total_seconds() / (365*24*3600)
2
3 if args.one_hour_step:
4     maturity_years = 1.0 / (252.0 * 6.5)    # ~1 trading hour
5 else:
6     maturity_years = t_to_expiry            # full expiry horizon

```

`investment_signal()` computes the relative mispricing and emits a trading signal:

$$\text{mispricing} = \frac{V_0^{\text{model}} - V_0^{\text{market}}}{V_0^{\text{market}}} \quad (16)$$

If this exceeds the threshold (default 5%), the signal is **BUY** (model says option is worth more than market price); if below $-\text{threshold}$, **SELL**; otherwise **HOLD**.

4. Case Study: META Platforms, Inc. (Put Option)

We ran the model on META stock with the following live inputs (as captured from the dashboard, 20 April 2026):

Parameter	Value	Parameter	Value
Spot S_0	\$670.91	Annualized σ	43.12%
Strike K	\$670.00	Maturity T (yrs)	0.04689
Risk-free r	5.00%	Option type	European Put
Expiry	2026-05-08	Vol window	60-day daily

4.1. Step-by-Step Calculation

Step 1 — Binomial tree factors:

$$u = e^{0.4312 \sqrt{0.04689}} = e^{0.09342} = 1.09788, \quad d = e^{-0.09342} = 0.91085$$

$$S_u = 670.91 \times 1.09788 = 736.58, \quad S_d = 670.91 \times 0.91085 = 611.09$$

Step 2 — Put payoffs:

$$P_u = \max(670 - 736.58, 0) = \mathbf{0.00}, \quad P_d = \max(670 - 611.09, 0) = \mathbf{58.91}$$

If META rises to \$736.58, the put expires worthless. If it falls to \$611.09, the put pays \$58.91.

Step 3 — Replicating portfolio:

$$\Delta = \frac{0.00 - 58.91}{736.58 - 611.09} = \frac{-58.91}{125.49} = \mathbf{-0.4694}$$

$$B = e^{-0.05 \times 0.04689} (0.00 - (-0.4694) \times 736.58) = e^{-0.002345} \times 345.73 = \mathbf{344.95}$$

The replicating portfolio is short 0.4694 shares and long \$344.95 in bonds.

Step 4 — Option price:

$$V_0 = (-0.4694) \times 670.91 + 344.95 = -314.94 + 344.95 = \mathbf{\$30.017}$$

Risk-neutral cross-check:

$$p^* = \frac{e^{0.05 \times 0.04689} - 0.91085}{1.09788 - 0.91085} = \frac{1.002346 - 0.91085}{0.18703} = \frac{0.09150}{0.18703} = 0.4892$$

$$V_0 = e^{-0.002345} [0.4892 \times 0 + 0.5108 \times 58.91] = e^{-0.002345} \times 30.088 = \mathbf{\$30.017} \checkmark$$

4.2. One-Period Binomial Tree

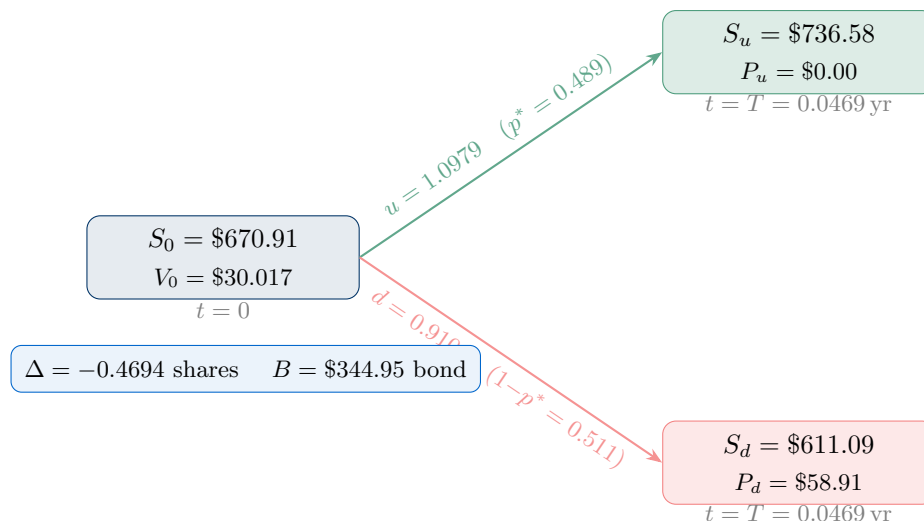


Figure 3: One-period binomial tree for META put option (K=\$670, expiry 2026-05-08).

4.3. Investment Signal

$$\text{Mispricing} = \frac{30.017 - 27.125}{27.125} = \frac{2.892}{27.125} = +10.66\%$$

Since $10.66\% > 5\%$ (threshold), the dashboard emits:

BUY option (model > market: potentially undervalued)

The model prices the put at \$30.017, while the market last traded at \$27.125, a 10.66% gap, suggesting the option may be undervalued.

5. Streamlit Dashboard

The interactive web application at binomial-model.streamlit.app wraps the entire pricing engine into a browser-accessible interface. The dashboard updates with live Yahoo Finance data on each run.

5.1. Left Panel — User Inputs

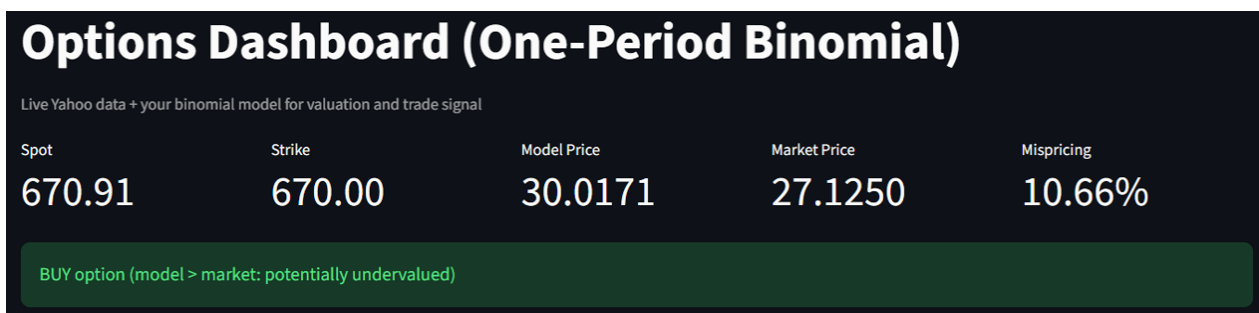
The sidebar exposes all model parameters via interactive widgets:

Section	Controls
Instrument	Stock ticker (searchable dropdown + free-text). Option type (call/put).
Model Inputs	Risk-free rate (decimal, $\pm$ buttons). Signal threshold (decimal).
Contracts	Number of contracts (integer). Contract size (e.g. 100 shares/contract).
Volatility	Mode: <i>daily</i> (historical close returns) or <i>intraday</i> . Daily history period (1y, 2y ...). Rolling vol window (slider, default 60). Intraday interval (1m) and lookback period (5d).
Horizon	Option expiry date (dropdown from live chain). Strike price (dropdown, auto-selects ATM). Toggle for 1-hour step vs. full expiry horizon.

5.2. Header Metrics

Five large KPI cards display: **Spot**, **Strike**, **Model Price**, **Market Price**, and **Mispricing (%)** — always in sync with the current ticker and settings.

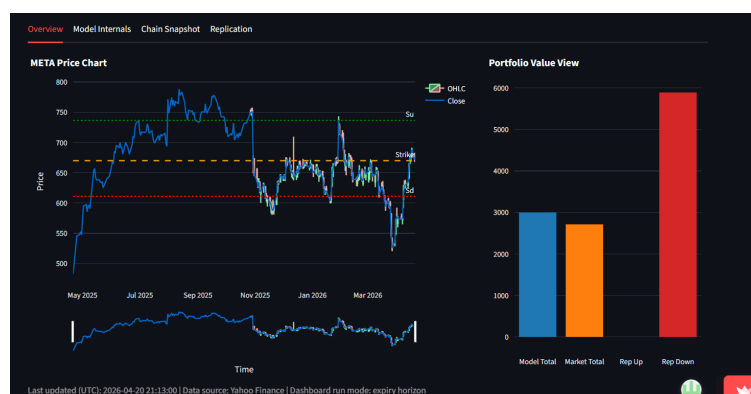
A colour-coded signal banner (green/red/grey) immediately communicates the trading decision.



5.3. Tab: Overview

META Price Chart — A dual-panel interactive Plotly chart showing the 1-year OHLC candlestick + closing price, with horizontal lines marking S_u (green), Strike (orange dashed), and S_d (red). A range-selector mini-chart sits below.

Portfolio Value View — A grouped bar chart comparing four values (at 100-contract scale): Model Total (\$3,001.7), Market Total (\$2,712.5), Replicating-Up portfolio (\$0), and Replicating-Down portfolio (\$5,890.5). The red bar (Replicating-Down) visually shows the magnitude of the put payout in the bearish scenario.



5.4. Tab: Model Internals

A structured bullet list exposes every intermediate model variable: symbol, option type, expiry, vol source, σ , T , r , u , d , S_u , S_d , payoffs, p^* , Δ , B , model price, market price, and threshold — all rendered in a monospace green font matching the META example values documented in Section 4.

Overview Model Internals Chain Snapshot Replication	
• Symbol: META	• Payoff Up/Down: 0.0000 / 58.9052
• Option: PUT	• Risk-neutral p: 0.489222
• Expiry: 2026-05-08	• Delta: -0.469420
• Vol source: daily close returns, window=60	• Bond B: 344.955342
• Sigma (annualized): 43.1234%	• Model option price: 30.0171
• Maturity T: 0.04689305 years	• Market option price: 27.1250
• Risk-free r: 5.0000%	• Decision threshold: 5.00%
• u, d: 1.097882, 0.910845	
• Su, Sd: 736.5800, 611.0948	

Last updated (UTC): 2026-04-20 21:13:00 | Data source: Yahoo Finance | Dashboard run mode: expiry horizon

5.5. Tab: Chain Snapshot

Displays the full option chain (puts or calls) fetched from Yahoo Finance for the chosen expiry, including strike, last price, bid, ask, volume, open interest, implied volatility, and in-the-money flag. A second table highlights the selected strike row ($K = \$670$ for META).

Overview

Model Internals

Chain Snapshot

Replication

	strike	lastPrice	bid	ask	volume	openInterest	impliedVolatility	inTheMoney
0	120	0.25	0	0.04	35	7	2.2969	☐
1	130	0.4	0	0.04	27	54	2.1875	☐
2	140	0.03	0	0.04	30	17	2.0938	☐
3	150	0.2	0	0.04	34	8	2	☐
4	160	0.03	0	0.04	25	47	1.9219	☐
5	170	0.03	0	2.41	None	110	2.8125	☐
6	180	0.21	0	0.2	5	110	2.0234	☐
7	190	0.2	0	1.68	49	55	2.4648	☐
8	200	0.03	0	0.2	28	44	1.8633	☐
9	210	0.19	0.01	0.15	225	69	1.7539	☐

Selected strike row:

	strike	lastPrice	bid	ask	volume	openInterest	impliedVolatility	inTheMoney
0	670	27	26.55	27.7	46	76	0.4616	☐

5.6. Tab: Replication

Surfaces the scaled replicating portfolio across the chosen number of contracts:

$$\text{Shares qty} = \Delta \times (N_c \times \text{size}) = -0.4694 \times 100 = -46.94$$

$$\text{Cash today} = B \times 100 = \$34,495.34, \quad \text{Portfolio value} = V_0 \times 100 = \$3,001.71$$

It also shows the replicating portfolio value in the up state ($\approx \$0$) and down state ($\$5,890$), confirming that the hedge perfectly tracks option payoffs.

Overview

Model Internals

Chain Snapshot

Replication

Contracts x Size

Replicating Shares Qty

Replicating Cash Today

Replicating Value Now

100

-46.9420

34495.5342

3001.7061

• Replicating portfolio in Up state: -0.0000

• Replicating portfolio in Down state: 5890.5246

• Cash grown to step: 34576.5991

• Model total value: 3001.7061

• Market total value: 2712.5000

Last updated (UTC): 2026-04-20 21:13:00 | Data source: Yahoo Finance | Dashboard run mode: expiry horizon

6. Results Summary

META Put Option — Key Results (20 April 2026)

S_0	\$670.91	u, d	1.0979, 0.9108
K	\$670.00	S_u, S_d	\$736.58, \$611.09
T	0.04689 yr	P_u, P_d	\$0.00, \$58.91
r	5.00%	p^*	0.4892
σ	43.12%	Δ	-0.4694
Model Price	\$30.017	B	\$344.95
Market Price	\$27.125	Signal	BUY (+10.66%)

The two pricing methods (replication and risk-neutral) agree to machine precision, validating the model's no-arbitrage consistency. The positive mispricing gap of 10.66% exceeds the 5% signal threshold, generating a BUY signal.

7. Limitations and Possible Extensions

The one-period model is deliberately simple. Several real-world factors are not captured:

- **Single period:** Real option pricing uses multi-period (or continuous) trees to capture the path-dependence of stock price dynamics.
- **Realized vs. implied volatility:** We use historical realized σ , whereas market prices embed *implied* volatility. The two can differ substantially (volatility smile/skew).
- **Dividends:** The model does not account for discrete dividends that would reduce the stock price at ex-dividend dates.
- **American options:** The framework prices European options; American options require backwards-induction through the full tree.
- **Transaction costs and liquidity:** The model assumes frictionless markets; real replication incurs bid-ask spreads and margin requirements.

Natural extensions include a multi-period binomial tree, Black-Scholes as the continuous limit, and a Greeks dashboard (Γ , Θ , Vega) within the Streamlit app.

8. Conclusion

This project successfully implements all four tasks of the Group 4 assignment. Starting from live Yahoo Finance data, the system computes realised volatility, constructs a one-period binomial tree, determines option payoffs, solves for the replicating portfolio (Δ, B) , and prices the option using both replication and risk-neutral methods, verifying their equivalence. The META case study demonstrates the full pipeline concretely: a put option with $K = \$670$ and 17 days to expiry is priced at \$30.017 by the model versus \$27.125 in the market, yielding a +10.66% mispricing and a BUY signal. The Streamlit dashboard makes all of this accessible interactively for any ticker and option contract, providing a practical, end-to-end financial engineering tool.

Data source: Yahoo Finance via `yfinance`. All prices as of market close 20 April 2026.